

MEAN Stack Machine Test - Task Management System

Duration: 1.5 - 2 hours

Stack: Next.js (Frontend) + NestJS (Backend) + MongoDB/PostgreSQL

Task Overview

Build a simple Task Management System where users can create, view, update, and delete tasks with categories.

Requirements

Backend (NestJS + MongoDB/PostgreSQL)

API Endpoints to implement:

1. POST /tasks - Create a new task
 - Body: { title: string, description: string, category: string, status: string }
 - Status can be: "pending", "in-progress", "completed"
2. GET /tasks - Get all tasks
 - Optional query params: ?category=work&status=pending
3. GET /tasks/:id - Get single task by ID
4. PATCH /tasks/:id - Update a task
 - Can update any field (title, description, category, status)
5. DELETE /tasks/:id - Delete a task
6. GET /tasks/stats - Get task statistics
 - Return count of tasks grouped by status

Database Schema:

Task:

- id (auto-generated)
- title (required, string)
- description (optional, string)
- category (required, string: "work" | "personal" | "shopping" | "other")
- status (required, string: "pending" | "in-progress" | "completed")

- createdAt (timestamp)
 - updatedAt (timestamp)
-

Frontend (Next.js)

Pages to create:

1. Home Page (/) - Dashboard showing:
 - Task statistics cards (count by status)
 - List of all tasks with filters by category and status
 - Each task card should show: title, category, status
 - Click on task to view details
2. Create Task Page (/tasks/new) - Form to create new task
 - Fields: title, description, category (dropdown), status (dropdown)
 - Form validation (title and category required)
3. Task Detail Page (/tasks/[id]) - View and edit task
 - Display all task details
 - Inline edit functionality OR edit button
 - Delete button with confirmation
 - Change status dropdown

UI Requirements:

- Use any CSS framework (Tailwind recommended) or plain CSS
 - Responsive design (at least mobile-friendly)
 - Loading states for API calls
 - Basic error handling (show error messages)
-

Submission Instructions

1. Create two separate folders: **frontend** and **backend**
2. Include **.env.example** files with required environment variables
3. Include README with:
 - Setup instructions
 - How to run the project
 - API endpoints documentation
4. Ensure the application runs without errors

Bonus Points (Optional)

- Add pagination to task list
- Search functionality
- Toast notifications for actions

Expected Deliverables

- Working NestJS backend with all endpoints
- Working Next.js frontend with all pages
- Database properly configured
- README with setup instructions
- Clean, organized code

Good luck!